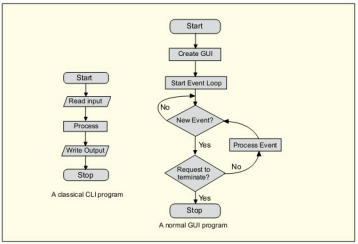

Figure 1: A comparison between CLI and GUI programs

In this example, the second number is accepted only after the first one. Once the second one is entered, the result emerges immediately. That is, the user cannot decide the sequence of the input and change its values after entering them. Besides, the program is terminated once the output is displayed.

Now think about designing the same program with a GUI. Let us put in two input boxes and a button for the result. The user can fill any input box first, i.e., the user can choose the order of input. The result is shown only if the button is pressed, so users have the freedom to edit their inputs. Further, the program is not terminated after showing the result, which means you can continue using it. Yes, all these features can be brought into command line programs, if required. But they are not available by default.

From this example, we can understand that GUI programs have an iterative nature. They start, present a lot of widgets, and wait until the user does something with them. On a user's action, an event handler is called to perform something, and if it is not to quit the program, it continues waiting for the next *event*. To put it in simple terms, a GUI program just sits and waits for events (that's why it is known as *event-driven programming*). This process continues until the program is asked to quit. Figure 1 illustrates a flowchart of this, compared to a classical CLI (Command Line Interface) program.

In practical programming, we design the interface using code or interface designers. The interface usually contains things like input boxes and buttons, which are known as *widgets* or *controls*. In the coding part, we write functions (modules of code) describing what should be done when the user does *this* with *that* widget. For example, we may write a function named *on_btn_add_clicked()*, which should be called when the user presses the *Add* button. Functions like this are known as *event handlers* (a.k.a. *callback* functions). After designing the interface and writing these functions, the task left to be done is to connect them. Each event handler is connected with a particular event of a widget, and once these connections are made, the program is ready to run.

The beauty of PyGTK

PyGTK is a GTK+ binding for Python. It is free (both freedom-giving and gratis), easy-to-use, and portable. PyGTK has become popular since it has the power and capabilities of GTK+, yet retains the simplicity of the Python language. It is suitable for both learning and professional programming.

PyGTK is shipped with almost all GNU/Linux distros. You don't have to install it. Windows users can download it from *pygtk.org*.

A 'Hello World' program

As usual, let's start with a 'Hello World' program. Our objective is simple—to show a window with a label displaying the text *Hello, World!* on it.

First, you have to know how to write and run Python code. There are many ways. Let's learn two basic methods that you can use on any platform.

Method 1: Using the command line

i) Python is shipped with almost all GNU/Linux distros. You don't have to install it. But if you are using another operating system, download and install it from *python.org*.

ii) Write your Python code using a text editor (e.g., gEdit on GNU/Linux and Notepad on Windows) and save it with a *.py* extension.

iii) Open a command line console (go to *Applications → Accessories → Terminal* on GNOME, use Konsole on KDE; if you are using Windows, press the *Win key+R*, type *cmd* and press the *Enter* key). Give a command in the following syntax:

```
<python_path> <code_path>
```

For GNU/Linux, type:

```
python /home/user/hello.py
```

For Windows, type:

```
C:\Python30\python.exe D:\hello.py
```

It works!

Method 2: Using an IDE

An IDE (Integrated Development Environment), as its name suggests, brings everything from the editor to the interpreter together. It has a built-in editor and *Run* button, which makes programming simple. You may install Geany or Stani's Python editor on GNU/Linux. Programs for Windows are also available on the Web.

Now let's come back to our 'Hello World' program. I'm directly jumping to the code. The explanation comes right after it.